

Documento Maestro de Arquitectura – v11

Reemplaza la versión 10.0. Cada afirmación de este documento fue verificada directamente contra el código y la base de datos en producción — no contra la especificación original.

Estado: en operación, verificado 2026-07-24 **Base de datos:** PostgreSQL / Supabase

Despliegue: Railway (backend) + Render (frontend)

00 Resumen ejecutivo

RadFor-360 es una plataforma SaaS de dos pilares — Radar (rastreo de convocatorias de financiación) y Formulador (estructuración de proyectos con IA) — con un pipeline de auditoría financiera en Pesos Colombianos construido y probado sobre datos reales. Este documento reemplaza la v10.0, que describía una migración a Firebase y un motor de búsqueda que no existen en el código real.

FRONTEND React + Vite + TypeScript puerto 5173	BACKEND Node.js / Express puerto 8000	BASE DE DATOS PostgreSQL (Supabase) pgvector + HNSW
AUTENTICACIÓN JWT propio (HS256) + fallback Supabase Auth	ALMACENAMIENTO Supabase Storage bucket privado "anexos"	DESPLIEGUE Railway + Render Dockerfile.backend/.frontend

01 Fe de erratas – qué cambió respecto a la v10.0

La v10.0 describía un estado aspiracional. Estas son las diferencias verificadas contra el código real:

AFIRMACIÓN EN V10.0	ESTADO REAL VERIFICADO
"Migrado a Firebase (Firestore y Auth), eliminando PostgreSQL y JWT manual"	falso — no existe ninguna referencia a Firebase/Firestore en el backend. La base de datos es PostgreSQL vía Supabase; la autenticación es JWT propio firmado con <code>JWT_SECRET</code> .
"Caché 24h vía Firebase/Redis"	falso — <code>tokenBlacklist.js</code> declara explícitamente "Sin Redis. Sin dependencias externas."
"Motor unificado en Gemini 1.5 con Google Search Grounding (reemplaza Perplexity, Tavily y Claude)"	falso — no hay ninguna referencia a Perplexity, Tavily ni Search Grounding. El Radar rastrea entidades reales vía <code>EntityScraper.js</code> + <code>CronScheduler.js</code> (ver sección Radar).
"Despliegue en contenedores Docker orquestado para alta disponibilidad"	parcial — sí existen <code>Dockerfile.backend</code> / <code>Dockerfile.frontend</code> , usados por Railway/Render como mecanismo de build. No hay una capa de orquestación propia (Kubernetes/Swarm) — son plataformas PaaS gestionadas.
"Consolidado a una única variable GEMINI_API_KEY"	impreciso — la variable real es <code>GOOGLE_API_KEY</code> , usada en 8 módulos distintos de IA. No está mal la intención (una sola clave de IA), pero el nombre no coincide.
RBAC (access_radar / access_formulador), aislamiento por org_id, hash de versión final	confirmado — coincide con el código real (<code>user_subscriptions</code> , middleware <code>requireAccess()</code> , <code>project_version_hashes</code>).

Añadido en esta versión: el pipeline de auditoría financiera (Sprints 1–4, ver sección "Pipeline de 4 motores") no existía en la v10.0 — se construyó, probó con datos reales y verificó en esta sesión.

02 Stack tecnológico real

FRONTEND

- React + TypeScript + Vite, servido en el puerto 5173 vía pm2 (`radar-frontend`).
- Code-splitting con `React.lazy()` para las ~25 páginas del ecosistema — reduce el bundle inicial.
- Diseño calcado desde Stitch MCP para pantallas específicas (marcadas explícitamente en el código con comentario "Calco estricto Stitch"); el resto son páginas construidas directamente en código.

BACKEND

- Node.js/Express, puerto 8000 vía pm2 (`radar-backend`), módulos ES (`import` / `export`) en todo el codebase.
- Motor de base de datos con 3 capas de resiliencia (`database.config.js`): pool directo (Capa 1) → REST de Supabase con `service_role` (Capa 2, la que opera hoy) → degradación controlada (Capa 3).

BASE DE DATOS

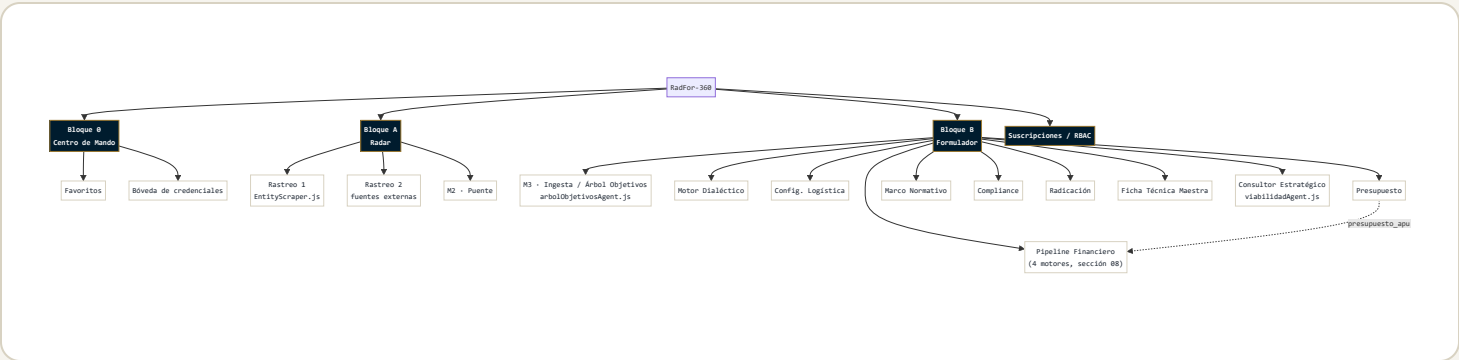
- PostgreSQL gestionado por Supabase, extensión `pgvector` con índices HNSW para búsqueda semántica (columnas `vector(768)`).
- Row Level Security habilitada en las 31 tablas públicas. **Nota honesta:** las políticas RLS son un blindaje secundario — el backend siempre opera con la llave `service_role` , que bypasea RLS por diseño. El aislamiento real ocurre en cada ruta vía `WHERE org_id = req.userId` .
- `proyectos.id` y `project_anexos.id` son `TEXT` (no `UUID`) — heredado del bootstrap original de `server.js` ; toda tabla nueva respeta ese tipo para poder referenciarlas.

DESPLIEGUE

- Backend → Railway, vía `Dockerfile.backend` + `railway.json` .
- Frontend → Render, vía `Dockerfile.frontend` + `render.yaml` .
- Almacenamiento de archivos → Supabase Storage (bucket privado `anexos`), no disco local — Railway reconstruye el contenedor en cada deploy, así que cualquier archivo en disco se perdería.

03 Organigrama del sistema

Jerarquía real de bloques → módulos → servicios. No es un organigrama de personas — es la estructura de responsabilidad del software.



04 Bloques y módulos

BLOQUE 0 – CENTRO DE MANDO

- **Gestor de Favoritos** — perfiles guardados del Directorio, persistidos en base de datos (no solo local).
- **Bóveda de credenciales** (`CredentialsPage.tsx` + `credentialVault.js`) — cada usuario puede configurar su propia clave de IA; si no la tiene, el sistema usa `GOOGLE_API_KEY` del servidor como respaldo.

BLOQUE A – RADAR (INTELIGENCIA DE MERCADO)

- **Rastreo 1** — scraping de entidades ya registradas en el Directorio (`EntityScraper.js`).
- **Rastreo 2** — fuentes generales externas (fundaciones internacionales: MacArthur, IDRC, IAF, Wellcome, Ford, OSF, Skoll, Omidyar, CIVICUS, GlobalGiving, NWO).
- Programado vía `CronScheduler.js` : Rastreo 2 a las 02:00, Rastreo 1 a las 02:30, backup a las 03:00 COT.
- Puente M2 → transfiere la convocatoria seleccionada al Formulador.

BLOQUE B – FORMULADOR (M3 A M12)

MÓDULO	ARCHIVO REAL
M3 — Ingesta / Árbol de objetivos	<code>arbolObjetivosAgent.js</code>
Motor Dialéctico	<code>motorDialectico.routes.js</code>
Configuración Logística	<code>configLogistica.routes.js</code>
Arquitectura Financiera (presupuesto)	<code>presupuesto.routes.js</code>
Marco Normativo	<code>marcoNormativo.routes.js</code>
Compliance	<code>compliance.routes.js</code>
Radicación	<code>radicacion.routes.js</code>
Reporte / Ficha Técnica Maestra	<code>reporte.routes.js</code> , <code>fichaTecnica.routes.js</code>

05 Suscripciones y control de acceso (RBAC)

Confirmado exacto contra el código real. Tabla `user_subscriptions` con columnas `access_radar` / `access_formulador` (booleanas). Middleware `requireAccess(modulo)` en el backend valida el permiso en cada ruta protegida; en el frontend, el componente `PlanGate` replica la misma lógica para no renderizar módulos sin acceso.

- **Suscripción Radar** → habilita Rastreo 1/2 + Directorio + Favoritos.
- **Suscripción Formulador** → habilita M3–M12 completos, incluida ingesta manual de pliegos propios.
- **Plan Suite** → ambos, más el puente M2 sin restricciones.
- Un usuario con solo Radar que intenta cruzar el puente es redirigido al panel de Planes; el botón valida el permiso *antes* de inyectar datos en el Formulador.

06 Agentes de IA – funciones reales

Todos comparten el mismo patrón de resiliencia: intentan Gemini primero, gateados por `geminiCircuitBreaker.js` (corta el circuito si hay error de cuota); si Gemini no está disponible, caen a una heurística o plantilla determinista — nunca lanzan una excepción, siempre devuelven un resultado utilizable con el mismo esquema.

AGENTE	FUNCIÓN	MODELO / FALLBACK
<code>viabilidadAgent.js</code>	Audita viabilidad del proyecto: escala/proporción (población vs meta), coherencia de anexos por taxonomía, veredicto VIABLE / VIABLE_CON_OBSERVACIONES / NO_VIABLE / RECHAZADO_INCOHERENCIA.	Gemini 2.0 Flash → heurística de reglas
<code>arbolObjetivosAgent.js</code>	Genera el Árbol de Objetivos (M3b): grafo Causas→Problema→Efectos invertido a Medios→Objetivo→Fines, en lenguaje MGA.	Gemini (google/generative-ai)
<code>enfoqueEntidadAgent.js</code>	Adapta la problemática/población del proyecto matriz al lenguaje de la entidad financiadora. <i>Nota: ruta backend activa; no confirmado si el frontend que la invoca sigue montado tras la eliminación de la ventana "Postulaciones".</i>	Gemini → plantilla determinista
<code>sectorClassifier.js</code>	Clasifica convocatorias del Radar por sector temático.	Gemini → mapa de palabras clave multi-idioma
<code>embeddingsService.js</code>	Convierte la Ficha Técnica a vector para búsqueda semántica (pipeline M7).	text-embedding-004, 768 dims
<code>markdownService.js</code>	Convierte PDF/DOCX/XLSX a Markdown (CLI Python), extrae campos estructurados de la convocatoria.	MarkItDown (Python) + Gemini

07 Relaciones entre módulos

- **Radar → Formulador:** el Puente (M2) transfiere la convocatoria seleccionada como semilla de un nuevo proyecto — requiere plan Suite.
- **Anexos → Pipeline financiero:** solo un anexo subido con `categoria = presupuesto_apu` y extensión `.xlsx/.xls` dispara `ExtractorService.js` → `AuditorForenseService.js` en cadena.
- **Pipeline financiero → Ficha Técnica Maestra:** hallazgos y métricas disponibles vía API; su inclusión visual en el reporte final es un paso de frontend aún no verificado.
- **Suscripciones → todo:** `requireAccess()` es el único punto de verdad.
- **Auth → Suscripciones:** `/api/auth/verify` devuelve el plan y los flags `access_radar` / `access_formulador` en el mismo payload.

08 Pipeline de auditoría financiera – 4 motores (nuevo)

Esta capa no existía en la v10.0. Construida, probada con datos reales end-to-end, y documentada en detalle esta sesión.

MOTOR 1 · AUTOMÁTICO

ExtractorService.js

Excel → `project_apu_lineas`, en COP, idempotente

MOTOR 2 · AUTOMÁTICO

AuditorForenseService.js

HSEQ ausente + descuadres aritméticos

MOTOR 3 · DELIBERADO

EstresadoFinancieroService.js

Choque macroeconómico, VIABLE/RIESGO/CRITICO

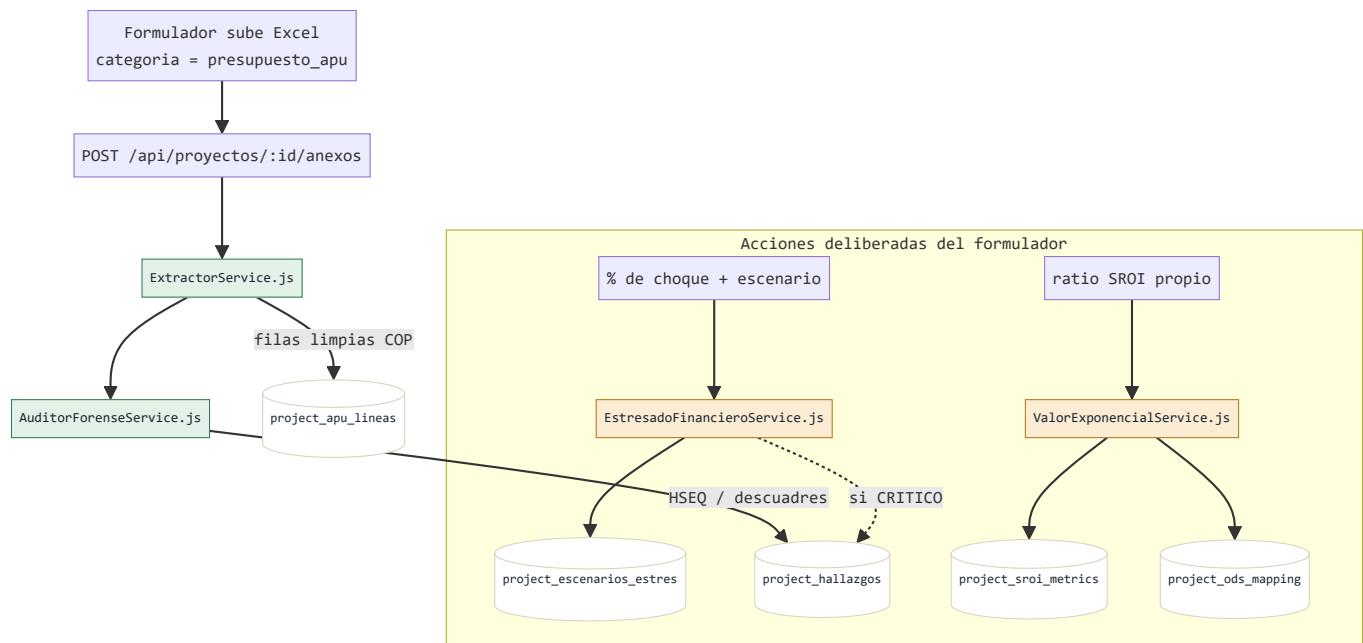
MOTOR 4 · DELIBERADO

ValorExponencialService.js

SROI (ratio explícito) + mapeo ODS

Principio de diseño no negociable: ningún motor fabrica una cifra financiera ni una cita legal. El etiquetado HSEQ es una alerta técnica, no un dictamen; el % de choque y el ratio SROI los define el formulador explícitamente — nunca un valor "estándar" inventado. El único número externo usado es el SMMLV 2026 real (\$1.750.905 COP, Decretos 1469/1470 de 2025).

09 Diagrama de flujo – Formulador



10 Radar – motor de rastreo real

No usa Perplexity, Tavily ni Gemini Search Grounding. Es scraping directo y estructurado:

- `EntityScraper.js` visita las páginas de convocatorias de cada entidad ya validada en el Directorio.
- `DataIngestor.js` normaliza lo extraído antes de insertarlo.
- `sectorClassifier.js` usa `GOOGLE_API_KEY` (Gemini) solo para clasificar sector, no para buscar.
- `CronScheduler.js` orquesta la periodicidad (Rastreo 2 → Rastreo 1 → backup).

11 Autenticación

- Login/registro propios en `server.js` : contraseña con `pbkdf2` (100.000 iteraciones, sha512), token JWT HS256 firmado con `JWT_SECRET` , expiración 7 días.
- `auth.middleware.js` valida primero contra Supabase Auth si está configurado; si falla, cae a la verificación JWT local.
- **Aprobación manual de registros (nuevo):** todo registro nuevo queda con `is_approved = 0` — no puede iniciar sesión hasta que un administrador lo apruebe desde `/admin/usuarios-pendientes` . Se dispara un correo de aviso al administrador vía Brevo.
- Modo Visitante: token temporal de 24h (`/api/auth/trial`), limitado a 3 tokens/hora por IP.
- Rate limiting estricto en `/api/auth/*` : 5 intentos fallidos por 15 minutos por IP.

12 Blindajes técnicos verificados

BLINDAJE	MECANISMO REAL
Aislamiento multi-tenant	<code>org_id</code> filtrado en cada ruta (no solo RLS, inerte bajo <code>service_role</code>).
Moneda única (COP)	Rechazo HTTP 422 si se detecta USD/EUR/GBP/CAD/MXN.
Idempotencia de presupuestos	Borra líneas de subidas anteriores con el mismo nombre de archivo antes de insertar.
Chunking	Inserciones en lotes de 500 filas.
Hash inalterable	<code>project_version_hashes</code> sella cada Ficha Técnica Maestra final.
Confirmación de borrado	Diálogo antes de "LIMPIAR" o eliminar cualquier fila con contenido en Anexos.

13 Deuda técnica real

- Envío de correo depende de credenciales de Brevo (configuradas parcialmente esta sesión); Resend (flujo de Stripe) sigue sin configurar.
- Stripe (cobros reales) sin credenciales de producción.
- Términos/Privacidad tienen campos pendientes (razón social, NIT) y requieren revisión legal.
- Exportación MGA/BID/OXI genera documentos basados en la estructura oficial, no son los formularios descargables exactos del gobierno.

RadFor-360 · ASFALTICA S.A.S. · Documento Maestro v11

Reemplaza v10.0 · Verificado contra código real, no contra la especificación